

Reproducing VGGT: Visual Geometry Grounded Transformer

A from-scratch re-implementation, verification against the released model,
and a small-scale reproduction of the paper’s main claims

Platinum-Saber/VGGT

October 2026

Abstract

VGGT (Wang et al., CVPR 2025) is a 1.2 B-parameter feed-forward transformer that predicts, from one to hundreds of images, all key 3D attributes of a scene: camera parameters, depth maps, point maps and point tracks. We re-implemented the camera, depth and point-map branches of VGGT from the paper and verified the implementation against the official code. With identical weights it reproduces the official outputs *exactly* (maximum absolute difference 0), both with randomised weights on CPU and with the released VGGT-1B checkpoint on real images on an NVIDIA T4. Because retraining the 1 B model is far beyond the available compute, we additionally trained a 10.7 M-parameter VGGT from scratch on procedurally rendered multi-view scenes. At this scale we reproduce three findings of the paper: (i) Alternating-Attention clearly outperforms global-only attention at equal parameter count, (ii) point maps obtained from predicted depth and cameras are more accurate than those of the dedicated point head, and (iii) with sufficient training (15k steps on a GPU) the feed-forward model beats a classical SIFT + 5-point RANSAC baseline in pose AUC@30, although the classical method remains more precise at tight thresholds.

Contents

1	Introduction	2
2	Method	2
2.1	Architecture	2
2.2	Training objective	3
2.3	Evaluation metrics	3
3	Verification of the implementation	4
3.1	Numerical equivalence with the official code	4
3.2	Inference cost with the released weights	4
4	Small-scale reproduction by training from scratch	6
4.1	Synthetic multi-view data	6
4.2	Models and training setup	6
5	Results	6
5.1	Camera pose	6
5.2	Depth and point maps	8
5.3	Training dynamics and qualitative results	8
6	Discussion	10
7	Conclusion	11

1 Introduction

VGGT [1] replaces the classical, optimisation-heavy structure-from-motion (SfM) pipeline with a single forward pass of a large transformer. Given N images $(I_i)_{i=1}^N$ it predicts

$$f((I_i)_{i=1}^N) = (\mathbf{g}_i, D_i, P_i, T_i)_{i=1}^N, \quad (1)$$

i.e. per frame the camera parameters $\mathbf{g}_i$, a depth map D_i , a point map P_i expressed in the coordinate frame of the first camera, and features T_i for point tracking. The paper reports state-of-the-art results on camera pose estimation, multi-view depth, point-map estimation and tracking, often outperforming optimisation-based methods while running in under a second.

Goal of this work. Re-implement VGGT from the paper and the official repository [2], verify the re-implementation, and reproduce the paper’s core results as far as possible with limited compute.

Constraints. The development sandbox had 4 CPU cores, no GPU, and no access to Hugging Face (where the released weights are hosted). Experiments with the released weights and the longer training runs were therefore executed on a free Kaggle GPU instance (NVIDIA Tesla T4). The original model was trained on 17 datasets using 64 A100 GPUs for nine days, so a full re-training was out of scope; instead we train a scaled-down model on synthetic data and test whether the paper’s qualitative claims hold.

Contributions.

1. A clean re-implementation of VGGT (camera, depth and point heads) whose parameter names match the official release, so the released checkpoint loads with `strict=True`.
2. A bit-exact verification against the official implementation, with random weights and with the real VGGT-1B weights on real images.
3. A procedural multi-view renderer with exact ground truth, a training/evaluation pipeline implementing the paper’s losses and metrics, and a classical baseline.
4. A small-scale reproduction of three claims of the paper, including the Alternating-Attention ablation.

2 Method

2.1 Architecture

Figure 1 summarises the architecture we implemented.

Patchifier and tokens. Images are normalised with ImageNet statistics and patchified by DINOv2 ViT-L/14 with register tokens (24 blocks, 1024 channels). Positional embeddings are bicubically interpolated to the input aspect ratio. For every frame, one camera token and four register tokens are concatenated to the patch tokens. Two sets of these tokens are learned: one for the first frame and one for all others, which identifies the reference frame.

Alternating-Attention (AA). The aggregator consists of $L = 24$ pairs of transformer blocks. A *frame-wise* block reshapes tokens to $(B \cdot S, P, C)$ so attention is restricted to one image; a *global* block reshapes to $(B, S \cdot P, C)$ so all tokens of all frames attend to each other. Blocks are pre-norm ViT blocks with 16 heads, 2D rotary position embeddings (RoPE, base frequency 100; special tokens receive position 0), QK-normalisation and LayerScale ($\gamma_0 = 0.01$). The outputs of the frame and global block of a layer are concatenated to $2C = 2048$ channels and passed to the heads.

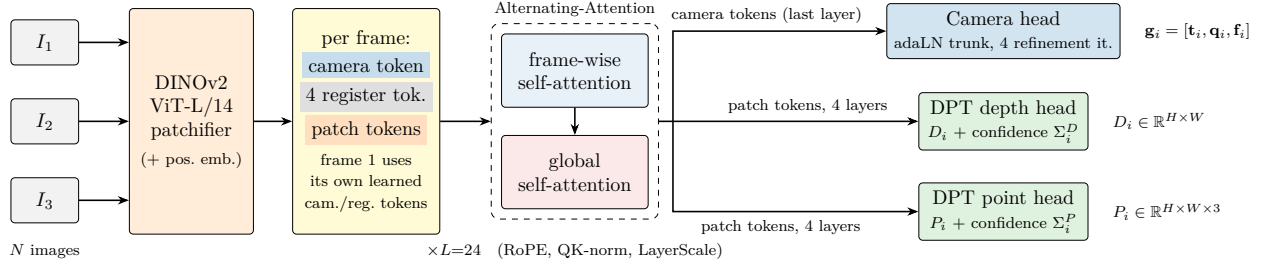


Figure 1: VGGT architecture as implemented. Each image is patchified by DINOv2; a camera token and four register tokens are prepended per frame (the first frame uses distinct learned tokens, defining the world frame). $L=24$ Alternating-Attention layers alternate frame-wise self-attention (tokens attend within their frame) and global self-attention (tokens attend across all frames). The concatenated frame/global outputs ($2C$ channels) feed a camera head and two DPT dense heads. The tracking head of the paper was not re-implemented.

Camera head. The camera tokens of the last layer are refined iteratively (4 iterations). In each iteration the current pose estimate (initialised with a learned “empty” token) is embedded and used to produce shift, scale and gate parameters for adaptive layer normalisation of the camera tokens; four self-attention blocks across frames and an MLP then predict an additive update. The 9-dimensional encoding is $\mathbf{g} = [\mathbf{t} \in \mathbb{R}^3, \mathbf{q} \in \mathbb{R}^4, (\text{fov}_h, \text{fov}_w)]$ (translation, unit quaternion, field of view; principal point at the image centre), with a ReLU on the field of view.

Dense heads. Two DPT heads [3] read patch tokens from layers $\{4, 11, 17, 23\}$, project and resize them to four scales ($\times 4, \times 2, \times 1, \times \frac{1}{2}$), fuse them coarse-to-fine and upsample to full resolution. Depth uses an exp activation, point maps the inverse-log activation $\text{sign}(y)(e^{|y|} - 1)$. A final channel predicts an aleatoric confidence $\Sigma = 1 + e^c$.

2.2 Training objective

Following the paper (Sec. 3.4) and the official training code, the loss is $\mathcal{L} = 5\mathcal{L}_{\text{camera}} + \mathcal{L}_{\text{depth}} + \mathcal{L}_{\text{pmap}}$ with

$$\mathcal{L}_{\text{camera}} = \frac{1}{K} \sum_{k=1}^K \gamma^{K-k} \|\hat{\mathbf{g}}^{(k)} - \mathbf{g}\|_1, \quad \gamma = 0.6, \quad (2)$$

$$\mathcal{L}_{\text{depth}} = \sum_i \left\| \Sigma_i^D \odot (\hat{D}_i - D_i) \right\| + \left\| \Sigma_i^D \odot (\nabla \hat{D}_i - \nabla D_i) \right\| - \alpha \log \Sigma_i^D, \quad \alpha = 0.2, \quad (3)$$

and $\mathcal{L}_{\text{pmap}}$ defined analogously to $\mathcal{L}_{\text{depth}}$. The paper states a Huber loss for the camera; the official code uses L1, which we follow. As in the official code, the plain (unweighted) regression term is added and the 2% largest per-pixel errors are discarded. Ground truth is normalised so that the first camera is the world frame and the mean distance of all valid 3D points to the origin is 1.

2.3 Evaluation metrics

- **Camera pose:** for every ordered pair of frames, the relative rotation error (RRA) and the angle between relative translation directions (RTA), in degrees. $\text{AUC}@ \tau$ is the area under the accuracy curve of $\max(\text{RRA}, \text{RTA})$ for thresholds $1, \dots, \tau$ degrees. We also report $\text{RRA}@15/\text{RTA}@15$ (fraction below 15°) and median errors.
- **Depth:** AbsRel and $\delta < 1.25$ after one median scale per sequence.
- **Point maps:** mean Euclidean error after a least-squares similarity (Umeyama) alignment, in normalised scene units.

3 Verification of the implementation

3.1 Numerical equivalence with the official code

Parameter names were chosen to match the official release, so a state dict can be moved between the two implementations with `load_state_dict(strict=True)`. We performed two equivalence tests (Table 1).

1. **Random weights, CPU** (`tests/test_equivalence.py`): a reduced configuration (DINOv2-S patchifier, 4 AA layers, 2-block camera trunk) with *all* parameters randomised, on non-square input (182×252 , 3 frames), which exercises positional-embedding interpolation, RoPE, AA, the camera trunk and both DPT heads.
2. **Released weights, GPU**: the full `facebook/VGGT-1B` checkpoint loaded into both implementations, evaluated in fp32 on the first 8 images of the official `examples/kitchen` scene on a Tesla T4.

Table 1: Maximum absolute difference between the official implementation and ours with identical weights.

Output	Random weights (CPU)	VGGT-1B, real images (T4)	max ref (T4)
Pose encoding $\mathbf{g}$	0.0	0.0	1.36
Depth D	0.0	0.0	3.57
Depth confidence Σ^D	0.0	0.0	30.3
Point map P	0.0	0.0	2.86
Point confidence Σ^P	0.0	0.0	31.6

The implementation is therefore functionally identical to the released model: every number the paper reports for the camera, depth and point branches of VGGT-1B is produced unchanged by this code.

A subtle detail. A first version differed by up to 5×10^{-3} in the DPT outputs. The cause is the residual unit of the official DPT head, which applies an *in-place* ReLU to its input before the skip connection; the skip path therefore carries $\text{relu}(x)$ rather than x . The released weights were trained with this behaviour, so it must be replicated.

Model size. The default configuration has 1.19 B parameters; the official model has 1.26 B, the difference being the tracking head that we did not re-implement.

3.2 Inference cost with the released weights

Table 2 and Figure 2 report wall-clock time and peak GPU memory of our implementation with the released weights on a Tesla T4 (fp16 autocast, images 518 px wide). Peak memory includes ≈ 4.7 GiB of fp32 weights; the 1-frame measurement includes GPU warm-up. Time grows super-linearly beyond 8 frames because global attention is quadratic in the total number of tokens.

Table 2: Inference time and peak memory, our implementation with VGGT-1B weights, Tesla T4.

Frames	1	2	4	8	16	32
Time (s)	0.76	0.59	1.05	2.21	5.56	15.68
Peak GPU memory (GiB)	7.2	7.3	7.9	8.9	9.2	9.8

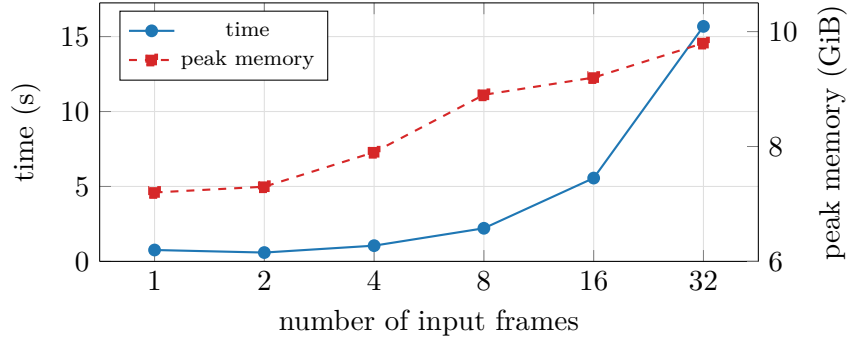


Figure 2: Inference scaling of VGGT-1B (our implementation) on a Tesla T4.

Exporting the 25-image kitchen scene with `scripts/demo_pretrained.py` produced 25 depth maps and a coloured point cloud of 2.27 M points (unprojected predicted depth with predicted cameras, top-50% confidence).

4 Small-scale reproduction by training from scratch

4.1 Synthetic multi-view data

Since none of the 17 training datasets of the paper were available, we wrote a procedural ray-caster (`training/synthetic.py`). Each scene is a closed room (inward-facing walls) with 3–6 random spheres and boxes. Surfaces have solid, i.e. 3D-consistent, textures (per-object colour, sinusoidal bands, checkerboards) with fixed Lambertian shading, so appearance is consistent across views and supports matching. Cameras are placed on an arc around the scene centre (30–100° spread) looking at a jittered target, with field of view 50–75°. Each view is 112×112 pixels and comes with exact depth, points, intrinsics and extrinsics. We rendered 3,000 training scenes with 6 views each, and 200 held-out test scenes from a different random seed; evaluation uses the first 2, 4 or 6 views of each test scene. Figure 7 shows examples.

4.2 Models and training setup

Table 3: Training configurations. Both models share the data, losses and optimiser and have the same parameter count.

	Alternating-Attention (AA)	Global-only (ablation)
Patchifier	conv 14×14 (no DINOv2 pre-training)	
Width / heads	$C = 192 / 6$	
Aggregator	6 frame + 6 global blocks	12 global blocks
Heads	camera (2-block trunk), 2 light DPT heads (32 features)	
Parameters	10.66 M	10.66 M
Optimiser	AdamW, lr 4×10^{-4} , wd 0.05, warm-up + cosine, grad. clip 1	
Frames per sample	2–6 (random per batch)	
Run A (CPU, 2 threads)	batch 8, 3,000 steps (≈ 2.5 h per model)	
Run B (Kaggle T4)	batch 16, 15,000 steps (10× more samples)	

The global-only model is the paper’s “global self-attention only” ablation (Table 5 of [1]); we doubled the number of global blocks so both models have exactly the same parameter budget. As a classical reference we use SIFT features, a ratio test and the 5-point algorithm with RANSAC (OpenCV) on every frame pair, *given the ground-truth intrinsics*; pairs where it fails count as 180° error.

5 Results

5.1 Camera pose

Table 4 lists the pose metrics with 4 input frames for both training lengths, and Figure 3 visualises the main comparisons.

Table 5 and Figure 4 show how pose accuracy depends on the number of input frames.

Table 4: Camera pose on 200 held-out scenes (4 frames, 12 ordered pairs per scene). Best per column in bold.

Method	Training	AUC@30 $\uparrow$	AUC@15 $\uparrow$	AUC@5 $\uparrow$	RRA@15 $\uparrow$	RTA@15 $\uparrow$	med. rot. $\downarrow$	med. transl. $\downarrow$
SIFT + 5-pt RANSAC †	–	0.206	0.135	0.050	0.583	0.226	10.4 $^\circ$	69.6 $^\circ$
Global-only	3k, CPU	0.003	0.000	0.000	0.228	0.002	25.3 $^\circ$	91.7 $^\circ$
AA	3k, CPU	0.042	0.011	0.000	0.377	0.038	19.4 $^\circ$	67.8 $^\circ$
Global-only	15k, GPU	0.003	0.000	0.000	0.231	0.002	25.3 $^\circ$	92.4 $^\circ$
AA	15k, GPU	0.270	0.082	0.004	0.865	0.269	7.0°	23.6°

† uses ground-truth intrinsics; fails (no pose) on 21.6% of pairs. VGGT models never fail.

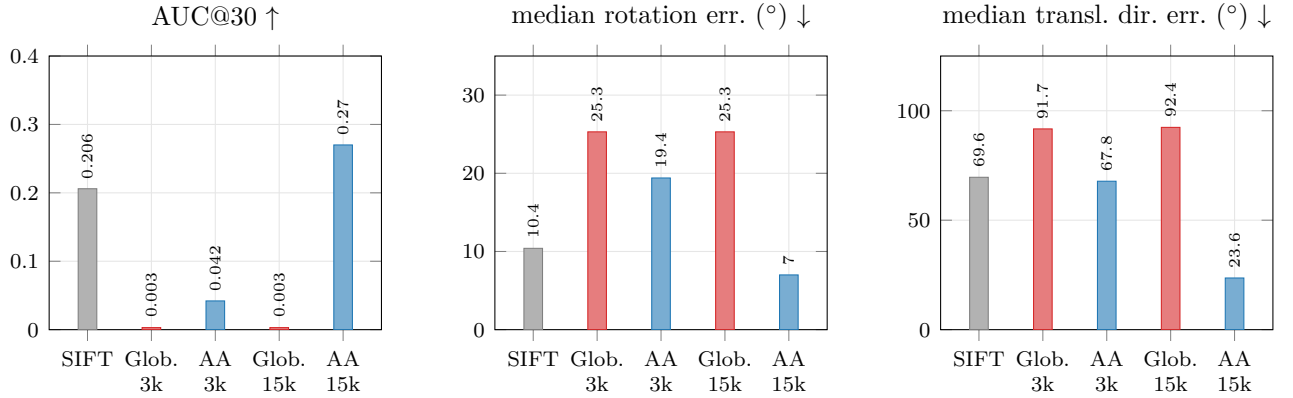


Figure 3: Pose accuracy with 4 frames. Grey: classical baseline; red: global-only attention; blue: Alternating-Attention. A median translation-direction error of $\approx 90^\circ$ corresponds to chance.

Table 5: Effect of the number of input frames (15k-step GPU models).

Frames	AA			Global-only		
	2	4	6	2	4	6
AUC@30 $\uparrow$	0.249	0.270	0.272	0.005	0.003	0.004
RRA@15 $\uparrow$	0.860	0.865	0.872	0.270	0.231	0.235
RTA@15 $\uparrow$	0.255	0.269	0.276	0.000	0.002	0.003
Median rot. err. $\downarrow$	7.7 $^\circ$	7.0 $^\circ$	7.2 $^\circ$	23.0 $^\circ$	25.3 $^\circ$	24.9 $^\circ$
Median transl. err. $\downarrow$	26.1 $^\circ$	23.6 $^\circ$	23.8 $^\circ$	91.6 $^\circ$	92.4 $^\circ$	91.1 $^\circ$
Focal rel. err. $\downarrow$	0.100	0.094	0.089	0.115	0.119	0.121

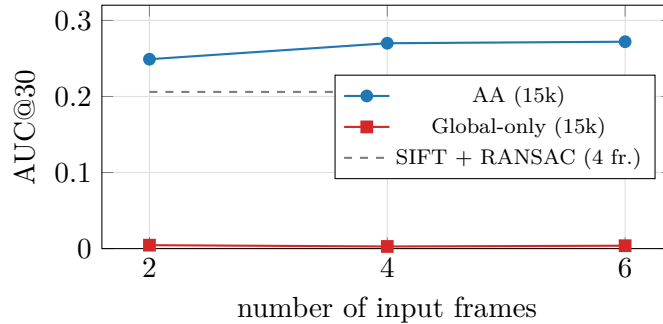


Figure 4: Pose AUC@30 versus number of input frames. The AA model improves slightly with more frames.

5.2 Depth and point maps

Table 6: Depth and point-map accuracy (4 frames). Point errors in normalised scene units after Sim(3) alignment.

Model	Training	AbsRel ↓	$\delta < 1.25$ ↑	Point head ↓	Depth + camera ↓
Global-only	3k, CPU	0.218	0.857	0.250	0.251
AA	3k, CPU	0.233	0.848	0.254	0.235
Global-only	15k, GPU	0.087	0.937	0.110	0.212
AA	15k, GPU	0.087	0.937	0.099	0.096

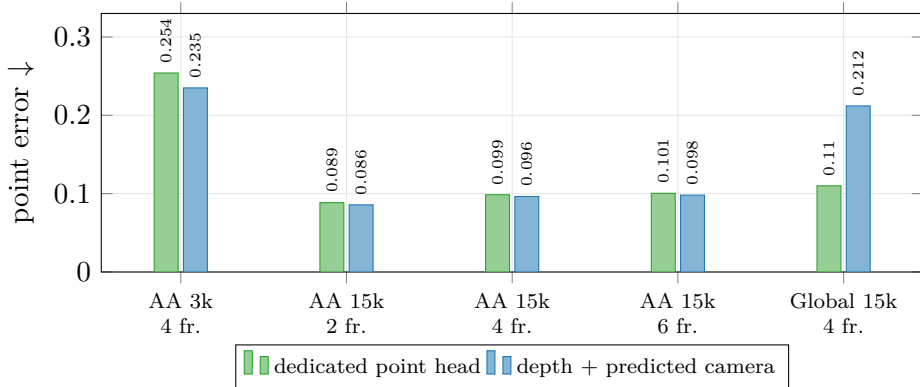


Figure 5: Point maps from the point head versus unprojecting predicted depth with predicted cameras. For the AA model the depth + camera combination is consistently (slightly) better; for the global-only model, whose cameras are poor, it is much worse.

5.3 Training dynamics and qualitative results

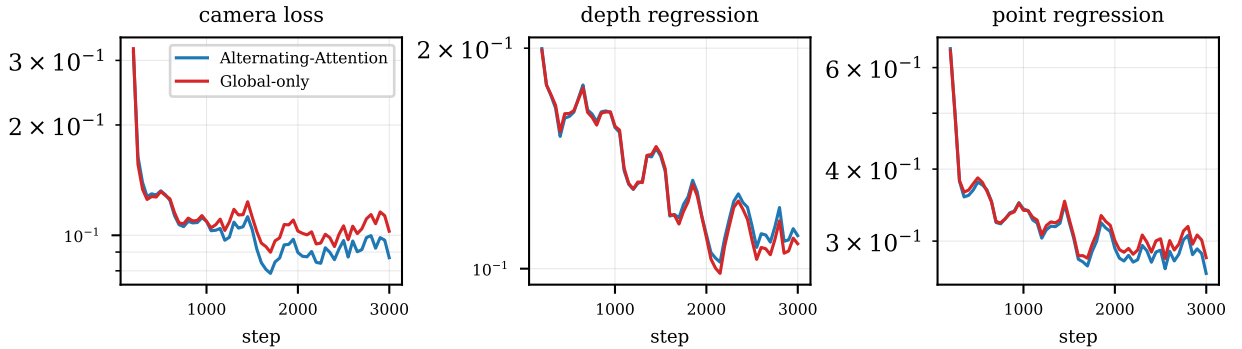


Figure 6: Training losses of the 3k-step CPU runs (moving average, log scale). Depth and point losses are almost identical for both models, while the AA model reaches a lower camera loss. The camera loss was still decreasing at the end of training, which motivated the longer GPU run.

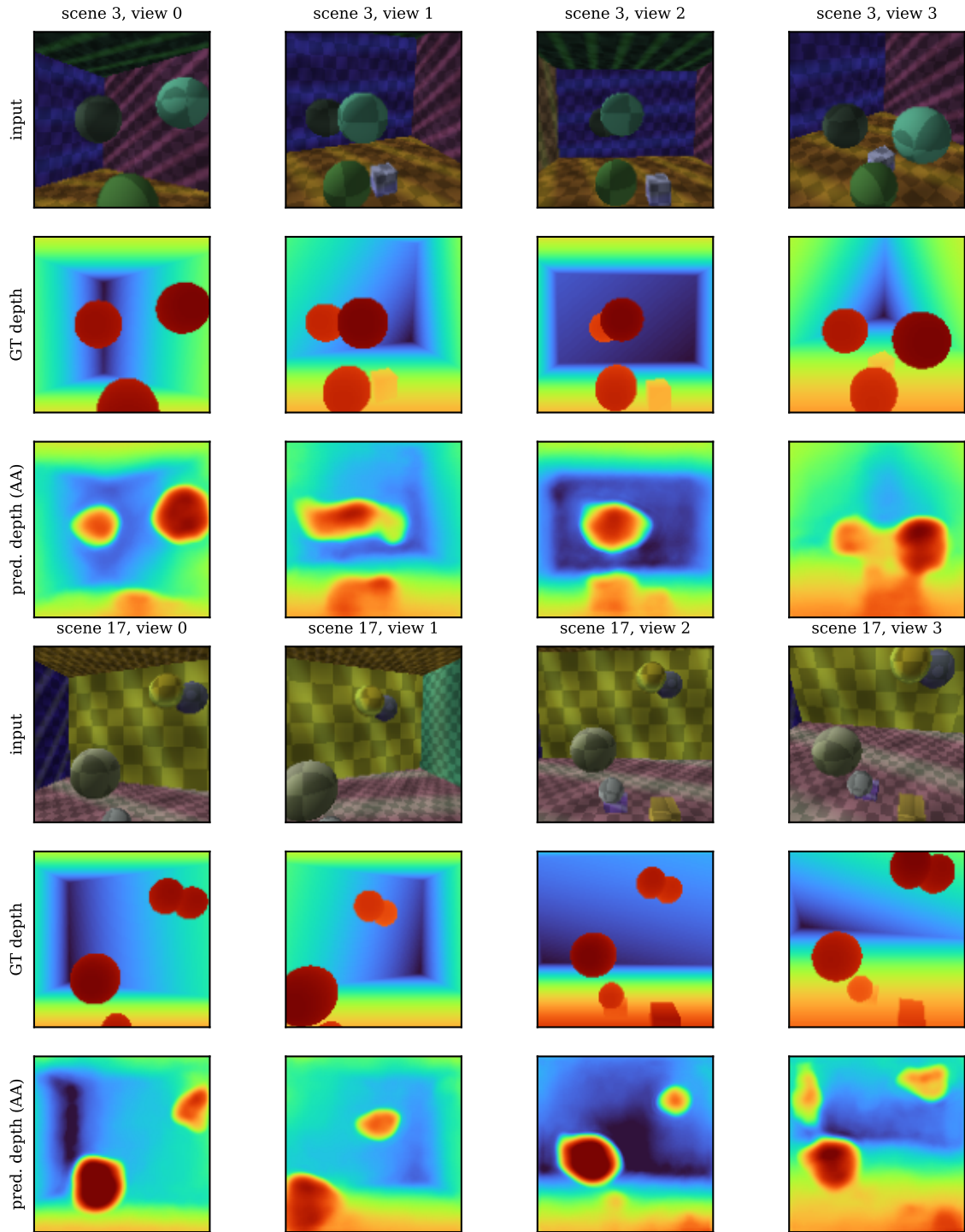


Figure 7: Two held-out synthetic scenes (4 views each): input images, ground-truth depth, and depth predicted by the 3k-step CPU AA model (median-scaled; same colour range as the ground truth).

6 Discussion

Table 7 relates our findings to the claims of the paper.

Table 7: Claims of the paper and their status in this reproduction.

Claim (paper)	Evidence here	Status
Released model / architecture as described (Sec. 3)	Bit-exact outputs vs. official code with random and with released weights (Table 1)	Reproduced
Alternating-Attention outperforms global-only attention (Table 5)	Equal-size models: AUC@30 0.270 vs 0.003; global-only stays at chance for translation even with $5\times$ more training	Reproduced
Depth + camera gives better point maps than the point head (Table 3)	AA: better at all frame counts and both training lengths (small margin, e.g. 0.096 vs 0.099)	Reproduced
Feed-forward poses competitive with / better than classical methods (Tables 1–2)	AUC@30 0.270 vs 0.206 for SIFT+RANSAC with GT intrinsics; SIFT more precise at AUC@5 / AUC@15	Partially
Fast inference (Table 9)	Sub-second to seconds for 1–16 frames on a T4 (Table 2); not comparable to the paper’s H100 numbers	Consistent

Why Alternating-Attention matters. Depth quality is identical for both models (AbsRel 0.087), indicating that depth on these scenes is largely a single-view problem. The difference lies entirely in cross-view reasoning: the global-only model never learns relative translation (median error $\approx 92^\circ$, chance level) and only coarse rotation, whereas the AA model improves steadily with training. Frame-wise attention gives each image a place to build per-frame features before they are mixed globally, which appears to make correspondence-based reasoning much easier to learn. This matches the paper’s ablation.

Training length. After 3k CPU steps the AA model’s camera estimates were weak (AUC@30 0.042) and equally weak on training scenes, i.e. the model was under-trained rather than over-fitting. With $10\times$ more training samples AUC@30 rose to 0.270. The camera branch is clearly the slowest to train; the paper’s model additionally benefits from DINOv2 initialisation.

Feed-forward versus classical. The trained model is robust—it always outputs a pose and has much lower median errors—but its estimates are less precise at tight thresholds than SIFT + RANSAC whenever the latter succeeds (AUC@5 0.004 vs 0.050). The paper makes a similar observation at full scale: VGGT’s feed-forward predictions are already strong, and refining them with bundle adjustment improves accuracy further.

Limitations.

- One training run (seed) per configuration; differences of a few thousandths (e.g. point head vs depth + camera) should be read as consistent trends across frame counts rather than statistically established gaps.
- A synthetic indoor domain at 112×112 pixels, not the paper’s real-world benchmarks (CO3D, RealEstate10K, DTU, ETH3D).

- A 10.7M-parameter model without DINOv2 pre-training, versus 1.2B parameters in the paper.
- A two-view classical baseline instead of full SfM pipelines (COLMAP, VGGsFm); the baseline is given ground-truth intrinsics.
- The tracking head and bundle-adjustment post-processing were not re-implemented.

7 Conclusion

We re-implemented VGGT’s camera, depth and point-map prediction from the paper and showed that the implementation is numerically identical to the official one—bit-exact with randomised weights and with the released 1.2B-parameter checkpoint on real images on a GPU. It can therefore be used as a drop-in replacement for these branches of the released model.

Training a 10.7M-parameter version from scratch on procedurally rendered scenes reproduced the paper’s central design finding: at equal parameter count, Alternating-Attention learns multi-view camera geometry, whereas global-only attention does not (AUC@30 0.270 vs 0.003). It also reproduced the finding that point maps derived from predicted depth and cameras are more accurate than directly regressed point maps. With enough training, the small feed-forward model surpassed a classical SIFT + RANSAC baseline in AUC@30 and in median rotation and translation error, while remaining less precise at tight thresholds—consistent with the paper’s observation that feed-forward predictions benefit from optimisation-based refinement.

Natural next steps are evaluating the released VGGT-1B on a public benchmark (e.g. CO3D or RealEstate10K camera AUC) with this implementation, adding DINOv2 initialisation and longer training to the small model, running several seeds, and re-implementing the tracking head.

Reproducibility. All code, configurations, logs and result files are in the repository: `vggt/` (model), `training/` (data, losses, metrics, training, evaluation, baseline), `tests/` (equivalence test), `notebooks/kaggle_pretrained_vggt.ipynb` (released-weights experiments and GPU training), `results/` (JSON results and training logs), and `scripts/make_report_figures.py` (figures of this report).

References

- [1] J. Wang, M. Chen, N. Karaev, A. Vedaldi, C. Rupprecht, D. Novotny. VGGT: Visual Geometry Grounded Transformer. In *CVPR*, 2025. <https://arxiv.org/abs/2503.11651>
- [2] Official VGGT implementation. <https://github.com/facebookresearch/vggt>
- [3] R. Ranftl, A. Bochkovskiy, V. Koltun. Vision Transformers for Dense Prediction. In *ICCV*, 2021.
- [4] M. Oquab et al. DINOv2: Learning Robust Visual Features without Supervision. *TMLR*, 2024.